

Multiservice billing system — a platform for the future

J Crookes

Billing systems are key competitive weapons for telecommunications companies. BT is developing a new generation billing system under the auspices of its Billing 90s programme. The term adopted in this paper for the system is multiservice billing system (MSBS). The paper outlines the strategic business issues which have shaped the design of MSBS. It describes the scale and complexity of the problem which makes the construction of a multiservice platform such a difficult feat of software engineering. The concept of a common product model, which underpins the system's design, is introduced. The later sections provide a brief outline of the architecture's physical realisation.

1. Introduction

The telecommunications industry is characterised by increasingly fierce competition on a global scale. Deregulation and growing customer sophistication have revolutionised the role of the carrier in the market-place. Technology is reforming the very nature of the industry as the boundaries between telephony, television and computers blur. These sweeping changes are placing carriers' traditional support systems under enormous pressure. Over the next decade this pressure will be directed heavily on billing systems. The days when billing was a simple back-office administration function are long gone. The complexities and scale demanded of a modern telecommunications billing system make its construction a most challenging feat of software engineering.

BT is developing a new generation billing system under the auspices of its Billing 90s programme. The term adopted for the system is multiservice billing system (MSBS). This paper discusses the design concepts which underpin MSBS and their physical realisation. Firstly, some strategic business issues [1] which have shaped the MSBS design are discussed.

1.1 Rapid product launch

Billing has to become an enabler, rather than a barrier, in the product-launch process. Traditional billing systems are almost invariably designed to support a particular product. Two main approaches to billing for new products are commonly used today — either a completely new billing system is constructed, or else an existing one is forced to bill products it was never designed to handle. The first course of action leads to a proliferation of small-scale

billing systems. The second approach places intolerable pressure on existing hub systems, causing their software to degrade at an alarming rate. The end result is the same in both cases — billing systems are notorious in the industry for inhibiting the launch of innovative products.

Much the same arguments can be applied to other operational support systems. However, billing has a unique link with product introduction. The demand from large customers for complete solutions is forcing carriers away from selling a relatively small set of fixed products. The new approach is called 'application selling'. Here carriers package various elements of service together to produce customised offerings — tailored either for specific industries (like transport, energy, or medical) or for particular business functions (e.g. LAN connectivity). The basic network building blocks remain the same; it is the billing system which wraps them with customised pricing, discounting, and reporting structures.

So the real competitive weapons for carriers are devices like volume-sensitive pricing, term contracts, contributory and eligibility (C&E) discounting, pool contracts, and flexible payment plans. It is the billing system — perhaps even more than networks — which provides the firepower for the competitive battles which lie ahead. If flexible pricing and reporting mechanisms are not available to hand, it will take months or years — rather than days or weeks — to construct most products. It is therefore the sophistication of the billing system's armoury which will largely determine who wins the war.

1.2 Financial drivers

The changes being experienced by telcos are taking place in the context of an unrelenting downward pressure on costs. The greatest cost reductions will come through the use of common technology and processes. BT currently has scores of different billing systems. The aim is to reduce these drastically. Cost can also be reduced by tackling errors in the various bill pipes. Revenue leakage and the direct costs associated with manual intervention clearly need to be minimised. Well laid out, easily understood, invoices will reduce the number of bill queries — which currently account for about 40% of BT's in-bound customer contact.

Financial drivers are not only about cost reduction. Third-party billing is a potentially lucrative service for carriers. For example, many large businesses employ significant numbers of staff to handle internal departmental accounting for communications. There is an opportunity to support the outsourcing of this function, provided that the telco's billing system can be driven to produce internal charging reports. The same approach can be applied within the carrier's own organisation. Some carriers have already adopted a retail/wholesale model, with financially accountable internal trading units. Initially the agreements between the various trading units will be at a fairly gross accounting level, but within a short period of time the trading units will demand operational systems which can implement genuine cost/price models. The danger for designers in all of this is that they start with too simplistic an abstraction of billing. Billing is not about managing the financial relationships between a single monolithic service provider and a set of customers. It models potentially recursive relationships between parties who may be service providers, customers, or both. The rate at which joint ventures and partnerships are springing up around the globe reinforces the point.

Perhaps the most exciting example of third-party billing relates to broadband intelligent networks [2]. Some form of third-party billing will be needed to manage the financial relationships between content providers, network operators, and customers (see Fig 1). So-called 'cybercommerce' challenges the traditional bill/pay paradigm used for the vast majority of telecommunications services. Singleton purchase — the most widespread financial model used on the Internet today — will not cope with high-volume usage services. However, other models exist — the pre-paid approach (based on electronic cash) or hybrids (where financial brokers manage credit agreements between customers and service providers) may one day supersede the old bill/pay method for some services. The carrier's role in this scenario needs to be positioned with extreme care — the implications which flow from debates about 'who will own debt on the Net' have profound consequences for business strategy. Billing engineers must therefore design systems which are flexible enough to handle the potentially

novel financial relationships between all the players in broadband intelligent networks.

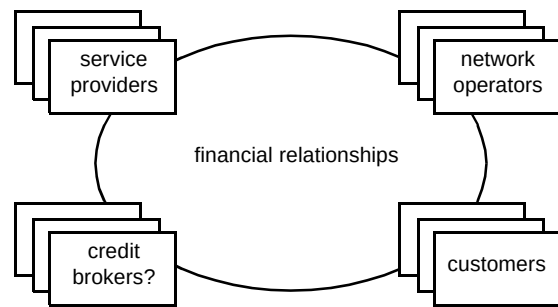


Fig 1 Players in broadband intelligent networks.

1.3 Customer management

Billing platforms of the future must be built as marketing systems. Their abilities to perform mass customisation, monitor revenue by segment, provide sophisticated messaging, and support modelling tools are as important as the administrative functions. Many of these requirements presuppose that the new billing platform will eventually handle all customer segments. The idea of a 'single solution' for all segments is a difficult presupposition for the engineer to digest — the characteristics of huge multi-national business customers seem so incredibly different from the characteristics of the millions of low-usage residential customers. The goal of common technology, while an admirable one, can sometimes be impossible to achieve. The argument against a single solution is a powerful one, given the inherent complexities of billing. However the advantages of a single solution are much greater than the cost saving which arises from common technology.

An analogy from the car industry might be helpful at this point. Most of the features which are taken for granted in mid-range cars today (like electric front windows, air bags, or power steering) were originally launched only in top-range cars. The features then 'trickled-down' to cars aimed at different customer segments. A similar trickle-down principle can be seen in the UK mobile market. It will also apply to features like consolidated billing, account codes, loyalty programmes, and customised packaging. A single billing platform for all customer segments allows this principle to be realised without the trauma of managing different implementations of the same product across disparate platforms. By far the most compelling argument for a single platform relates to 'affinity products'. The millions of residential PSTN customers are not completely autonomous beings with no relationships to each other or to the businesses in which they work. Affinity products will begin to weave increasingly complicated relationships between groups of residential customers, and between businesses, to the point where the cost-centre hierarchy of a large multi-national will not seem unduly complex. The

argument for a single solution for all segments therefore appears to be sound. (The term 'single solution' refers to common technology. It does not imply that a single operational platform is required. MSBS technology will be deployed in a variety of configurations for different suppliers and customer segments.)

Market-support billing systems cannot be divorced from customer information warehouses [3]. Large customers require a holistic view of their inventory and usage across all functional areas and product lines. Account managers also need this view to monitor revenue trends and to look for new sales opportunities. In the residential market the effects of promotions on the call behaviour of certain customer types need to be analysed. A properly designed billing system will provide a rich source of up-to-date information for marketing. The design should therefore decouple the billing applications from the information which they produce. And this information should be easily available to marketing analysis systems. Indeed the billing platform should have as its foundation an operational warehouse which holds inventory, usage and financial reports.

The classical problem with any operational warehouse, however, is the quality of the customer data it holds. Large customers have complex cost centre, service management, and legal organisational structures. The difficulties in tracking change are considerable. In addition, most order-taking front-ends do not impose sufficient discipline on the user community. Perhaps the greatest cause of poor data quality stems from the way users' success is measured. BT's user communities have traditionally been targeted on metrics like speed of order taking. Data quality has not been a measure of success. If market-support billing systems are to be deployed successfully, some radical process re-engineering is needed. Systems should allow customer data quality to be measured so that the incentive schemes used to drive users can include data quality metrics. The MSBS design envisages customer data quality centres which support migration, reconciliation and quality audits of customer data, as shown in Fig 2.

The MSBS warehouse, shown in Fig 2, is only one of the components which make up BT's overall operational customer database (OCDB). The treatment of OCDB is beyond the scope of this paper.

2. Scope

Billing is the bloodstream of the business. Like any vascular system, it cannot be isolated from the other functional areas which it pervades. Billing is linked to customer handling, order handling, portfolio management, market analysis, fraud detection and service repair. The advent of intelligent networks has added complexity to the relationship between billing and the network. All these interdependen-

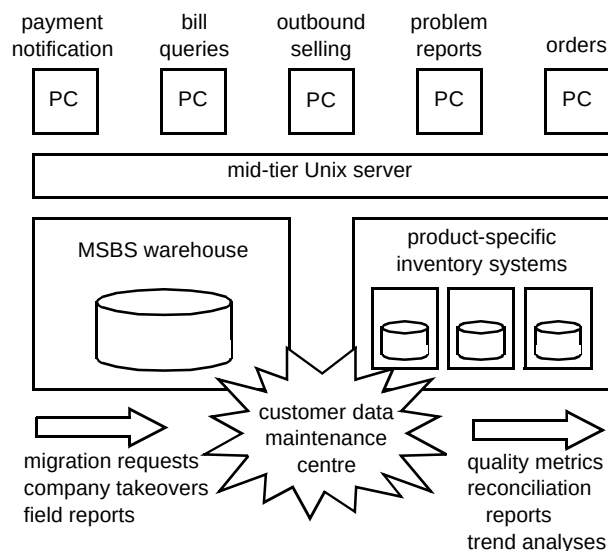


Fig 2 Customer handling.

cies make it very difficult to define the scope of billing. The problem is not solved purely by architectural divisions of responsibilities — agreeing the dependencies between billing and other development programmes is a vital management responsibility. The following sections discuss some of the key scoping problems.

2.1 Customer handling

Billing front-office activities should be regarded as a functional 'partition' of a general-purpose customer-handling system. Prerequisite functions like customer identification, workflow management and general contact management are needed by billing — but are not specific to billing. Hardware platforms and internal networks are also shared between billing front-office applications and other customer handling functions. MSBS meets these requirements by aligning with BT's 'three-tier' front-office architecture. The three tiers are termed presentation, business logic and data services [4]. The tiers are implemented on PC, Unix server, and back-end data server systems, as shown in Fig 2.

The implementation problems introduced by common infrastructure are considerable. If a prerequisite function exists (such as customer identification), then inter-programme agreements must be formed for billing to use the function. These agreements must flush out any subtle differences in priorities and apportion costs between the two programmes. If the functions do not exist, or if they are so embedded within a legacy system as to be unusable, then billing designers have no option but to build tactical solutions for non-billing functions.

Most good billing systems will provide a range of bill delivery mechanisms — paper, EDI, magnetic tape, CD ROM, and so on. This technology is another candidate for

common infrastructure. A bill is just a specialised type of customer report, so the traditional 'back-end' of billing should be regarded as a general customer-report delivery mechanism. The argument is architecturally correct and points to the need for more reuse agreements between programmes. However, the MSBS design team judged that the system could not be delivered if too many interdependencies between development programmes existed. This specific issue illustrates a more general point — even so-called target designs must be deliverable. Perhaps the most difficult decisions made by the MSBS design team have not been purely technical — the skill levels of suppliers and operators had to be taken into account; the design had to create self-contained work units for which suppliers could be held clearly accountable; and sometimes perceived technical risks had to be tackled as if they were real technical risks. The design process is not simply a scientific debate about abstractions and technology. The managerial art of actually delivering massive software systems must also shape the design.

2.2 Order handling

The most obvious link between order handling and billing is inventory. It is useful to divide inventory into two categories — instances of billing products, and (for the most part) instances of non-billing products such as leased circuits and PSTN lines. The MSBS design accepts that for many years to come the master inventory for non-billing products will reside on product-specific order handling systems. It is possible to view these various physical databases as one single logical database, tempting the architect to build the billing applications on top of a cross-reference table which points to all the individual physical databases. However, it is BT's view that primary billing requires a physical database of consolidated inventory for both billing and non-billing products. The reasons largely relate to performance and exception management. In the single logical database scenario the billing applications would have to gather inventory from many different databases for each customer and then convert the inventory into a standard format. Given that the bill run will be multi-streamed, the roll-back implications in the event of a link or database failure would be extremely complex. The single physical database solution is not without its complexities either. Controlling the replication of inventory data for non-billing products requires application-specific protocols.

Another relationship between order handling and billing relates to quotations. At some stage during the order-taking process customers will be given an estimated price for the service they are ordering. If several options are available then 'what-if' pricing becomes necessary. A quotation is one of the four possible times when pricing is required in any of the telco's business processes, the others being real-time pricing in the network, initial pricing for usage limits

and validation, and bill-time. MSBS uses a single pricing component which operates in all four modes.

The last major link between order handling and billing relates to the construction and maintenance of cost-centre hierarchies for large customers. MSBS is designed to offer secure Intranet-type access to customers, and also provide account managers with lap-top support systems which allow cost-centre structures to be edited at the customer's premises. These edited structures can then be downloaded into MSBS through fixed or mobile links. The lap-top field-support systems are also equipped with the pricing component described earlier to allow the account manager to walk through competitor comparison and 'what-if' scenarios with the customer.

2.3 Portfolio management

A key characteristic of the MSBS platform is its support for multiple services. It is a general-purpose suite of software components [5] which can be data driven to implement a particular product. Data driven systems do not provide a panacea for the problem of complexity. The complexity problem is not removed; it is simply moved from code to data. The logic which formally describes BT's current portfolio of products is contained in millions of lines of Cobol and C code. Over time these products will be implemented on MSBS. Their logic will then be expressed (in a much more abstract and coherent manner) in terms of rules and data — but the logic is obviously still required. New business processes and user roles are needed to create and maintain this business logic. In traditional systems the process of formalising the product description was performed by software engineers. An equivalent role needs to be performed by people who can formally describe product logic in terms of abstract MSBS rules and data. The sort of 'service-creation environment' where BT's portfolio of products will be described is a system called the portfolio database (PDB).

PDB is primarily a product repository, but it is better described as a total service creation environment for BT and its partners [6]. As Fig 3 shows, PDB provides a set of workbenches which allow the order handling, billing and intelligent network logic of a product to be designed. This logic is then cascaded, for example, to the MSBS platform which interprets the rules and data. The scope of PDB is clearly quite enormous and is not implementable as a single programme. MSBS is acting as the pioneer of the new generation operational support systems which will be driven by PDB. The pace at which the full PDB vision is implemented will be dictated by the service management and IN development programmes.

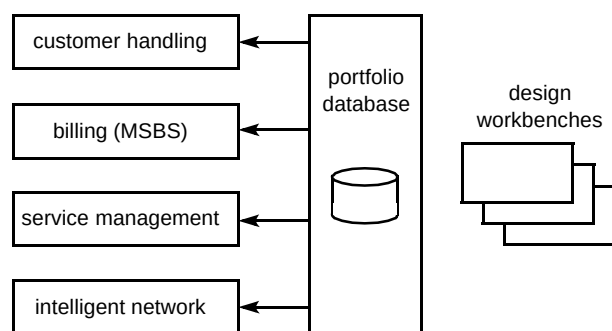


Fig 3 The portfolio database.

2.4 Network

The historical interface between networks and billing has been event collection. Primary data generators (PDGs), such as switches, generated potentially chargeable events. A batch file mechanism later collected them for billing. A multi-service billing system needs to collect events from a large number of different PDGs — from various switch vendors, from service control points and from service configuration and monitoring systems. MSBS supports the concept of network mediation which involves the translation of the PDG-specific event formats into a common data description language.

Network interconnection and network intelligence have had even more significant effects on the billing/network interface than mediation. Network interconnection has created a large number of wholesale agreements in the UK between BT and other licensed operators (OLOs). These agreements will become more sophisticated as deregulation allows wholesalers (including BT) to treat OLOs as customers. The recursive nature of billing is seen here as each event's wholesale and retail costs need to be apportioned between the operators and the customer.

The concept of an intelligent network (IN) has been discussed for decades, and although the original narrow-band vision might now seem outmoded, there is no disputing the move of intelligence from network elements to computer systems. In the IN world, MSBS will be responsible for real-time forward pricing, real-time monitoring, and 'hot' billing. The problems associated with late calls is a classical problem for real-time pricing; if volume-related pricing or usage limits are required in real time, then late or duplicate usage events are exceptionally difficult to handle. For these reasons MSBS draws the distinction between pricing for advice or transaction payment and pricing for billing.

These sorts of issues highlight the danger of starting with POTS (plain old telephony) as the model interface between billing and networks. In fact it is a pernicious model which ignores the need for solutions to things like

event routing, personal numbering and electronic cash management.

3. The engineering challenge

3.1 Scale

A visit to one of BT's print and despatch centres provides an excellent demonstration of the scale of its billing operation. Forklift trucks lift pallets of enveloped bills into Royal Mail trucks lined in order of postcode. For PSTN alone BT currently processes over 100 million usage events each day. At any one time there are over 5 billion events in its 6000 network elements and support systems. BT has over 23 million customers ranging from single-line residential customers on low-usage schemes to large multi-nationals with over 40 000 individual accounts. When full itemisation was first introduced a BT business customer in London had their bill delivered in two fully loaded vans (they now use CD ROM as their preferred media). BT's postage costs are enormous so there are customised discounts with the UK Post Office based on the weight of envelopes. Even small changes to the bill format which require another page can therefore force the bill over a weight limit and add millions of pounds on to the operational cost of billing.

BT currently bills PSTN using its customer service system (CSS). CSS produces over 430 000 bills each night and takes over 41 000 orders each day [7]. It runs on 30 mainframes, each supporting over 4000 terminals. There are 4443 on-line screens. The MSBS design has to cope with the eventual migration of all customers off CSS. So the volumetrics of the system are enormous. A weekly full image copy of all unbilled usage data will be about 1 TB. There will eventually be 24 x 7 service coverage with no maintenance schedules and automatic back-ups. Disaster recovery times for all critical customer data is 24 hours.

3.2 Complexity

Billing systems are very expensive to develop because they are so complicated. Even to process customers with the simplest of products requires quite substantial logic. Consider some of the normal everyday situations which affect billing for a residential customer. The customer may order more products and update or cease existing ones during the bill period. BT may also revise the terms of its products. OfTel might decide to schedule a mass number change (such as Phoneday), while the Government may announce a change to the rate of VAT. If the customer moves house an on-demand bill which copes with broken-period rentals might be requested. Two years later the customer might query this old bill and ask BT to recalculate it.

In many ways a billing system is like a time machine, because at any point the product definitions, customer inventory, tax regime, or network numbering scheme could change. The BT portfolio of products is not a two-dimensional snapshot of the products on offer today. It is a three-dimensional graph which describes the relationships between all past, present and future products. The need for a historical dimension also applies to customer inventory and taxation regimes. All these structures need to be merged together to form a customer-specific graph. Billing can thus be viewed as a particular moment in time when a customer's charges are pushed through this graph.

The problem of time becomes much more complicated when dealing with large customers — their cost-centre structures change frequently, and it can take up to six months to negotiate a customised deal (i.e. the agreement must be applied retrospectively). More generally, the bill packages for large customers might contain over twenty different reports — each giving different breakdown analyses for the current and previous bill periods. The results can be shown graphically with customised text.

Complexity also arises from the number of products which the billing system must implement. BT has in the region of 100 000 product codes to manage. Converting these product codes into a more coherent and streamlined set of MSBS products is a difficult analysis task.

MSBS is designed to be a global billing system. Bills for large multi-national customers must deal with global tax regimes, multiple currencies, and multiple languages. (US tax is particularly difficult as the various federal, state and local regimes must be applied at individual event level.) If the technology is to be used by many partners around the world, then the user interfaces must also be language-sensitive.

MSBS is not only a multiservice billing system. It is a multisupplier system which will act as a third-party billing platform. It will therefore have to support the product sets of BT's partners and customers. The six complexity factors discussed above are only a sample of the underlying issues which conspire to make billing systems so complicated.

3.3 *Process re-engineering*

BT divides its activities into a set of generic business processes, three of the main ones being [8]:

- gain business,
- launch/withdraw products and services,
- bill and collect.

Over time, MSBS will have a radical effect on many of these processes. The changes will alter or create brand new

user roles. Nearly 70 of these roles have been identified during the business process analysis phase of MSBS. They range from pricing modellers and compensation managers, through to credit referrals analysts and international settlements managers. Billing production analysts, data integrity managers, bill designers and tax analysts are other examples of new or altered user roles.

The introduction of any large-scale computer system must involve process re-engineering. A common mistake in managing these sorts of projects is for the business and technical decision makers to agree on high-level business requirements and priorities. The technical community then creates a solution based on these requirements and existing processes. So-called 'detailed field implementation' issues are often left to people in the organisation who do not have sufficient authority to change user roles and business processes. So historical anomalies are propagated through to the new systems — often at considerable expense — simply because the implications of the original requirements on processes had not been dealt with at the outset of the project.

3.4 *Income integrity*

The unique characteristic of any billing system is the financial significance of its data. MSBS will eventually be responsible for billing and receiving payments for billions of pounds of revenue. The controls and measures which ensure the integrity of this revenue form the foundation of MSBS. The requirements for income integrity are all concerned with accuracy, timeliness, authorisation, validity, traceability, availability and completeness.

MSBS implements financial controls at three levels:

- **in-process** controls are activated as part of the normal execution of each software component,
- **out-of-process** controls are implemented across a set of components but are nonetheless part of the overall system's normal operation,
- **external** controls include everything from procedures to quality management systems.

MSBS models the major flows of data through the system as a set of 'pipes' as shown in Fig 4. Each section in the pipe is a stateless application component. Work units are constructed and pushed down the pipe. Each component applies its own in-process controls and then lets the work unit carry on down the pipe. Out-of-process controls analyse the state of the work units at each pipe 'joint' and in the secure data stores to assure integrity.

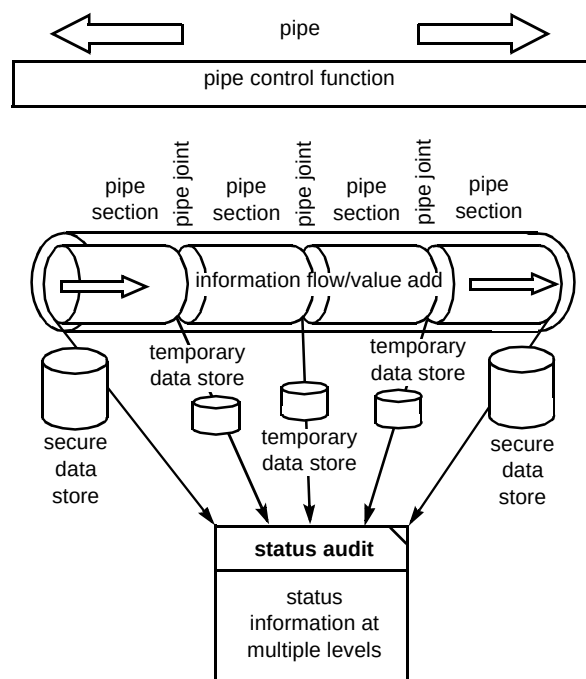


Fig 4 The pipe concept.

4. Design concepts

4.1 Product model

MSBS is premised on the idea that all telco-related products can be described using a common framework or model. The question 'What is a product?' has no simple answer. The MSBS product model is concerned only with describing product behaviour and attributes which are of financial significance. For example, the process of making a telephone call involves a financial transaction so the attributes associated with the event (such as start and stop time, or called number) will be modelled. However, an off-hook operation is not modelled as it has no monetary significance. The product model has two levels: a logical view and a physical implementation view. The following paragraphs describe each view in turn.

At the logical level, the product model introduces the concept of 'domains'. Figure 5 shows the four major domains which have been created — billing, service management (covering order handling, work management,

and service monitoring), customer handling, and the intelligent network. The modelled behaviour of a product is segmented between these domains. The domain concept means that (for MSBS) only the billing domain needs to be modelled in detail. The other domains are only of significance insofar as they generate potentially chargeable events. All events are conceptually held in a super information base (SIB).

Object-oriented (OO) techniques are used at the logical level. An inheritance tree has been designed for BT's portfolio. (For example, ISDN and PSTN share all the characteristics of a public switched service, leased circuits and flexible bandwidth services also have inherited characteristics, and all discounting products — whether they be simple, retrospective, forward, volume-incremental or volume-step — can be viewed as particular sub-types of a C&E structure.) The basic building blocks within each domain are called product elements. Product elements are objects in the sense that they encapsulate both data and behaviour. The data is modelled as attribute lists and the behaviour is described using rules. A syntax allows the conditions and actions which make up a rule to be linked. Rules are invoked by external operations. During the execution of a rule, events may be generated. Events, like product element data, are described as attribute lists.

To begin to see how these abstractions work together, the first requirement is to understand how events are generated and then how they are processed. In the case of a simple residential phone, events will be generated when the exchange line is installed and subsequently when the phone is used. An exchange line is made up of two sub-product elements: external and internal wiring. An 'order' operation in the order-handling domain will cause an 'install' operation to be invoked in the service-management domain. This operation will invoke a rule or other operations. The chargeable event raised during this process will flow back into billing. Its attribute values will show whether or not internal wiring had to be put in place. When a 'use' operation is invoked in the network, an event (i.e. a traditional PSTN call record) is created. The attributes in this event will contain all the possible factors which might affect the charges that billing will raise when processing the event.

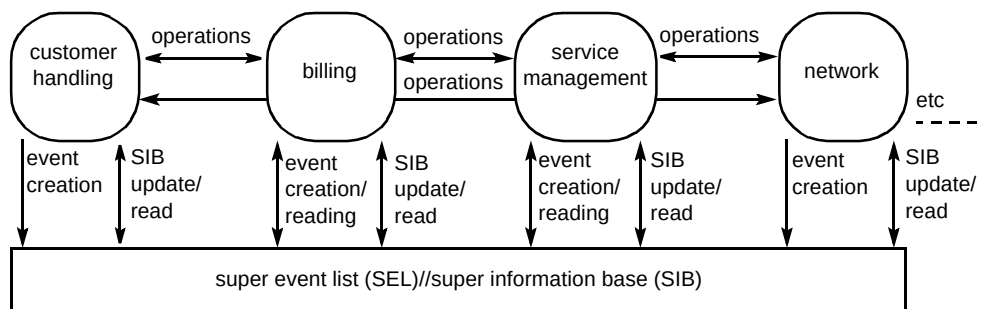


Fig 5 MSBS logical domains.

The PSTN service is much more than the exchange line. The basic pricing, bill reporting and discounting structures are all modelled as product elements in the billing domain. Network tariffing, discount products and the bill format are examples of elements which contain rules and data. The rules in these elements select the events generated in other domains (such as installation and usage events) and process them. Processing might involve sorting events, adding new attributes to them, or even creating new elements such as an instantiated report. The rules in the billing domain can themselves generate events for things like recurring charges.

The modelling of even simple products involves rigorous analysis to ensure that all the financial events and rules are covered. Fortunately the work to date has revealed that the inheritance concept leads to significant reuse of product elements.

The model's abstractions (elements, operations and rules) are very similar — but not identical — to pure OO concepts like objects, messages and methods. OO vocabulary is deliberately not employed because, although the logical design uses OO throughout, the implementation does not. MSBS is not an object-oriented system. It is what is termed 'object-based'. A process has been designed which takes the logical OO product definitions, and translates them into a form which — while retaining data encapsulation [9] — externalises the behaviour and removes inheritance. OO implementations only make sense when permanent OO storage is available. The software industry's experience of building OO applications on top of relational databases is an unhappy one. Furthermore, the view was that no OO database existed which could handle the volume and throughput requirements discussed earlier. It is true that some OO databases present some very impressive laboratory benchmarks. But raw performance is only one of many criteria used to select the best storage technology. Operational features like catalogue management and archiving need to mature before a database is ready to handle terabytes of data.

So the MSBS product model consists of an object-oriented logical model, an object-based physical implementation, and a manual process which translates the logical to the physical.

4.2 Protocols

The concept of 'pipes' has already been introduced. Pipes have been defined for order entry, receipting, main usage entry and the bill run. Other pipes define the controlled flow of data from external billing systems, to despatch centres and to revenue reporting systems. Pipes have value because they use a layered protocol, as shown in Fig 6.

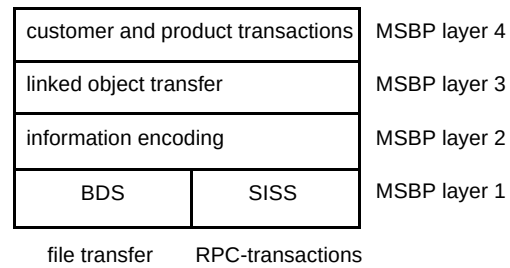


Fig 6 Multiservice billing protocol (MSBP).

The protocol is termed multiservice billing protocol (MSBP). Layer 1 provides both managed file transfer and transaction-oriented interconnect. The balanced distribution service (BDS) manages the despatch and receipt of files between the various physical machines in the system. It provides a range of monitoring and configuration tools for the automation platforms and operators in the command centres. The systems interconnect service shell (SISS) is a bespoke transaction-oriented protocol which provides an RPC-like function between MVS/CICS and Unix processes. It was decided that the distributed computing environment (DCE) was still too slow for the response time targets to be met. However, SISS has been designed to be replaced by DCE once it matures.

MSBP layer 2 deals with how MSBS information will be encoded. The platform independent record (PIR) is a simple transfer syntax. PIRs are nested type, length, value structures which can be parsed and built by both Unix boxes and MVS mainframes. MSBS will always control the definition of its interfaces, so there is no need for the protocol to distinguish between abstract and transfer syntaxes. It is an architectural principle that the parsing of PIRs should be performed by a shared software component called the generic parser module (GPM). This code is being used by all the MSBS components to ensure consistency and to eliminate the replication of PIR handling code.

GPM is portable to systems with an ANSI C compiler. To allow the GPM to be used by mainframe components, a mainframe application programming interface (API) or 'wrapper' has been written on top of GPM, using Cobol, as illustrated in Fig 7.

MSBP layer 3 uses the encoding capability of layer 2. Its protocol structure is shown in Fig 8. Layer 3 carries two basic types of data — an 'object download' or blocks of (layer 4) product and customer data. MSBS stores its data in relational databases. However, the physical implementation of the product model is still 'object-based'. In database terms this means that encapsulated data 'objects' are held in fifth normal form, encoded in the PIR syntax. Some of the attributes are normalised out, but most are just blocks of raw data as far as the database is concerned. The real advantage

of this approach is that new products can be launched without any need for a database restructure.

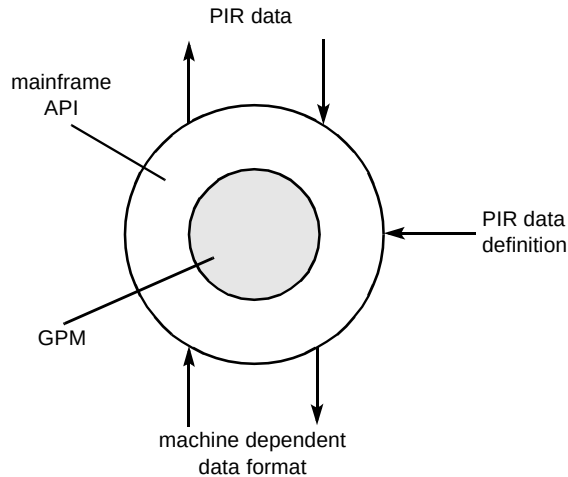


Fig 7 Generic parsing module.

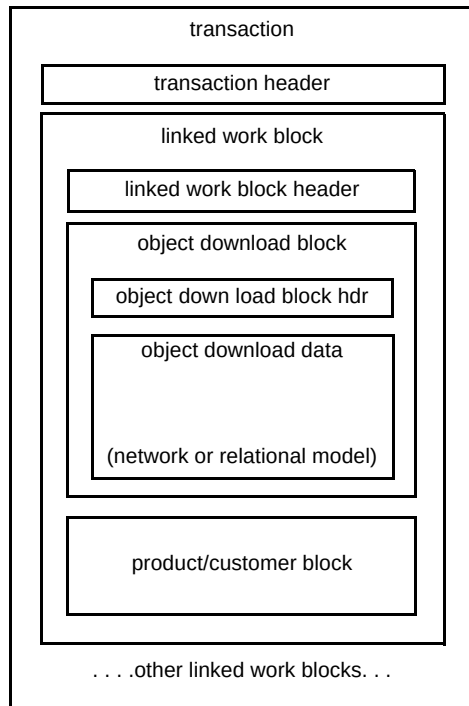


Fig 8 MSBP layer 3.

MSBS has to manage the replication of this data between different physical processors. Sometimes heterogeneous databases need to be kept in step (between the warehouse and the legacy systems' inventories, for example); sometimes slave processors need copies of data from master repositories in order to perform their tasks. In the latter case, the replicated data will be loaded directly into shared memory as an in-core network database. MSBP

layer 3 is designed to handle all these situations. It can be viewed as a database 'in-flight', although it must always be remembered that the protocol is deliberately specific to MSBS applications and its encoding syntax. The entire information base can be downloaded, or (for efficiency) individual CRUD (create, read, update, delete) methods can be applied to specified objects. If the receiver uses a network database, then link records are included in the object block. Link records can refer to objects outside the object block, so MSBP control mechanisms have to handle cases where external references are unresolved.

The primitives or data units provided by layer 4 within MSBP are defined directly by the product model. They are all PIR encoded:

- products,
- product elements,
- operations on product elements,
- events,
- rules,
- inventory.

One advantage of a layered protocol is that the various controls and measures which must be applied in pipe sections and joints can be defined in terms of these layers. At the most basic level checksums ensure that the bits sent into a pipe come out the other end. At the more abstract levels, financially significant controls can be applied, e.g. the total revenue of events in each work unit can be monitored.

4.3 Engines

Many of the application components which form pipe sections have special characteristics. They are stateless in the sense that no checkpointing or roll-back is performed within the application. The work is broken down into sufficiently small units to allow each application to make a fresh restart in the event of a major failure. These applications all use data which is in shared memory. Data movers transfer the required data to and from secure storage before and after the applications run. The applications use and process three types of data — rules, events, and customer inventory (see Fig 9). The rules are downloaded from the PDB system, and are interpreted by the applications. Any application within MSBS which conforms to these principles is called an engine.

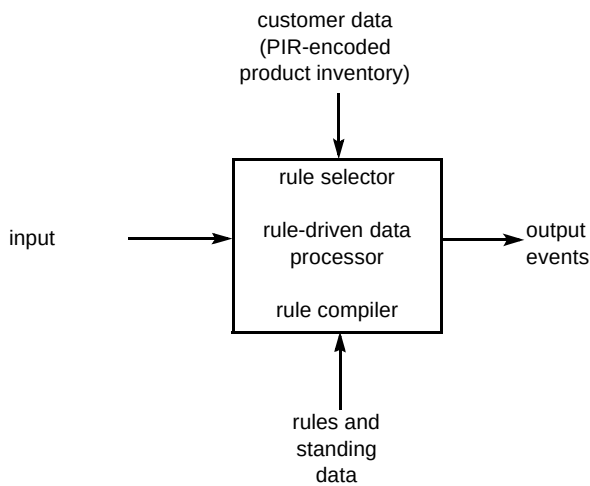


Fig 9 Concept of an engine.

Engines may perform their function as part of mass processing within a scheduled window, or they may be required to process a single request from a user. They must be scalable. Where engines are running in parallel each will deal with a complete logical unit of work.

4.4 Rules

The main engine in MSBS is called the pricing and assembly engine (PAE). It is responsible for all bill-time pricing, discounting, taxation, and grouping of events for reporting. Its major function is to calculate the customer's financial liability. The PAE provides a good illustration of a 'rules-driven' engine. Consider a hypothetical cross-product discount package called 'Home & Away' launched on 1 January 1996. The product is a C&E discount across mobile and fixed PSTN lines. Provided that the combined mobile and fixed usage spend in a bill period is greater than £150, then a symmetrical tiered discount is applied as follows — the greater the spend on mobile usage, the greater the discount off fixed line rental; and correspondingly, the greater the spend on fixed usage, the greater the discount on mobile rental. Imagine that the marketing department wishes to apply a flat-rate discount to the fixed rental and a percentage discount to the mobile rental. This product is placed in the BT portfolio on top of mobile and PSTN pricing products. (In the example, shown in Fig 10, these products have been allocated codes P00001 and P00002 respectively.)

The rule which the PAE interprets to implement this product is shown in Fig 10. The macros at the start simply provide meaningful names to the columns of the table which defines the actual tiers of the discounts. The working data and results are loaded on to the wagons of a datatrain which is pulled through the rule sequence.

The TransferEventsCost function puts the total cost of the selected events on to the datatrain. It is an example of the

basic functions implemented on the engines that can be linked together with specific data to form rules. The engines have an extendible library of event selection, event preselection, transfer and path-selection functions, which form the basic vocabulary for product description. The design of these functions was perhaps the most critical aspect of the whole of MSBS. As the engines process millions of events on each invocation, the efficiency of list processing is paramount. The functions must also be at the correct level of abstraction — they should mirror the business problem which they solve. (Some so-called generic systems have been shipwrecked by creating a vocabulary which is so general that it is little different from a programming language such as C. The logic in such systems ends up as product-specific as the code on traditional platforms. The only difference is that the logic is now inefficiently interpreted rather than compiled.)

5. Physical design

The MSBS physical architecture [10, 11] is shown in Fig 11. Some of the components have already been outlined — the portfolio database (PDB) and the three-tier billing front-office partition (BFOP) of PCs, mid-tier Unix system (called the customer reception system — CRS) and the back-office data servers.

The design is centred on the customer billing processor (CBP). The CBP is an MVS/DB2 platform. Many of the billing applications which run on the CBP are engines which produce the actual invoices, and deliver them to the print factories. The pricing and assembly engine, which is the main engine, actually runs on adjunct symmetric multiprocessor Unix machines, linked to the CBP by high-speed FDDI (fibre distributed data interface) channels. (There will be a 'farm' of two or three PAEs for each CBP.) The CBP is also responsible for receipting and debt management. All the CBP applications are entirely separate from the MSBS data warehouse, which is called the customer information warehouse (CIW). There can be many physical instances of the CBP/CIW. The CIW holds both inventory and usage for a particular set of customers, but uses internal directories to provide data transparency. The usage is collected from the network by network mediation processors (NMPs). The usage events are routed onward to service domain processors (SDPs) which perform initial pricing and validation. The SDPs load the usage events into the appropriate CIW.

The management and control of the processing components is the responsibility of the management and control architecture (MCA) discussed in section 5.4. The hub of the MCA is the management and control bridge (MCB).

```

macro Start = 1996/01/01:00.00.00.000;
macro mobile_spendColumn = 1;
macro fixed_flatrateColumn = 2;
macro fixed_spendColumn = 3;
macro mobile_percentColumn = 4;

```

```

table discount Table = { /*

```

date	mobile_spend	fixed_flatrate	fixed_spend	mobile_percent */
Start	0.00..10.0000	-1.00	0.00..10.00000	-0.010},
Start	10.00999..50.00999	-5.00	10.00999..50.00999	-0.50},
Start	50.00999..100.00999	-10.00	50.00999..100.00999	-0.10},
Start	100.00999..150.00999	-15.00	100.00999..150.00999	-0.150},
Start	150.00999..	-20.00	15.00999..	-0.200}

```

};

```

```

rule Discount =
TransferEventsCost { XFER(1), XFER(10), (Usage and (event.PSC="P00001")), false) ->
TransferEventsCost { XFER(1), XFER(20), (Usage and (event.PSC="P00002")), false) ->
TransferOperation (XFER(30), XFER(10), XFER_ADD, XFER(20) ->
TransferExpression (XFER(40), 150.00) ->
PathSelectOnCompare (XFER(30), XFER_Greaterthan, XFER(40)) ->
[
DiscountEvents ((Recurring and (event.PSC="PSC00001")),
DISCOUNT_FLATRATE_OF_CURRENT, DiscountTable, TRANSFER,
XFER(20), mobile_spendColumn, OUTPUT, fixed_flatrateColumn)
->
DiscountEvents ((Recurring and (event.PSC="PSC00002")),
DISCOUNT_PERCENT_OF_CURRENT, DiscountTable, TRANSFER,
XFER(10), fixed_spendColumn, OUTPUT, mobile_percentColumn)
|
NullFunction
];

```

Fig 10 Example product implementation rule.

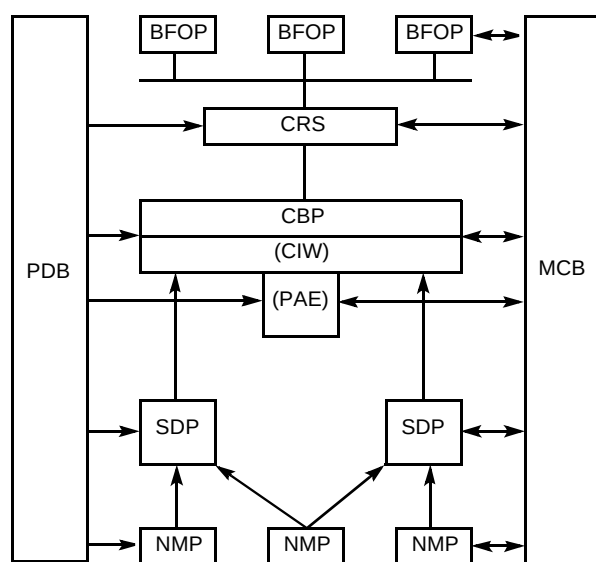


Fig 11 MSBS overall physical design.

clearly defined responsibilities for event routing. The NMP is responsible for a set of network elements; the SDP is responsible for a set of features (such as a range of virtual private network (VPN) numbers); the CBP is responsible for a set of customers. In the POTS world, both NMP and SDP operate in batch mode on large Unix SMP machines. In the intelligent network, the NMP and SDP components can run as portable interactive modules on IN platforms. The SDP, PAE, and CRS all share the same pricing engine software.

Figure 11 only shows the core billing components. The CBP interfaces to order handling systems, BACS, bulk-receipting centres, debt collection agencies, other billing systems, MIS and general ledgering systems. The full range of front-office capabilities that MSBS supports is not shown in Fig 11. A complete description of the design is beyond the scope of this paper.

108 All the components communicate using MSBP. The three main processors — NMP, SDP, and CBP — have

The following sections provide a brief overview of each major component in MSBS.

5.1 Customer billing processor

The CBP is the heart of MSBS. Its internal design is shown in Fig 12. The CBP is composed of four layers — a data layer (the CIW), a utilities layer, the main application layer, and the control layer.

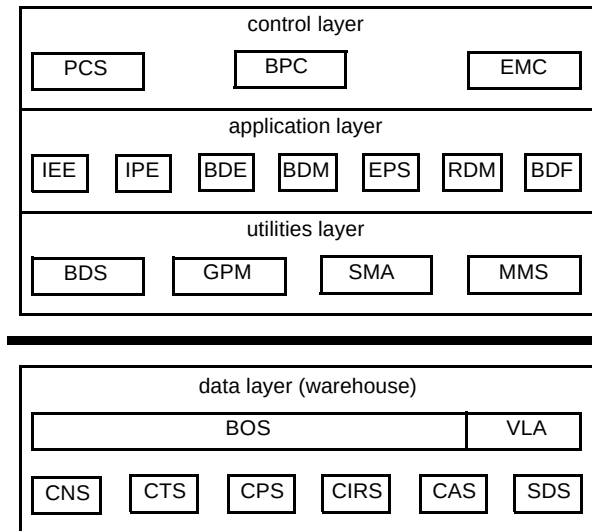


Fig 12 Customer billing processor.

The CIW is made up of six servers. The customer name server (CNS) holds all names and addresses used throughout the system. The CNS is BT's definitive operational names database. The transaction server (CTS) holds all the terabytes of usage data. It holds the other event types as well: recurring, one-off, payment, and SLA violation events. The CPS holds all product data. Like many of the other servers, the data is held in fifth normal form.

A brief explanation of how MSBS products are ordered is necessary at this point. The PIR-encoded events and product elements are instances of the definitions held in PDB. They are created by BT's order handling system called COSMOSS. The PDB system drives the automatic generation of screens on COSMOSS for each product in its repository. So when a 'Family and Friends' product is defined in PDB, for example, the COSMOSS screens which capture the set of five numbers are automatically generated. The captured attributes are stored as PIR-encoded product elements in the CPS.

The customer invoice and reporting server holds previously constructed invoices. The customer accounting server (CAS) holds information generated by the receipt and debt management application. The support data server holds copies of PDB data, and a variety of other standing data. All these servers are accessed through the business object server (BOS). BOS presents a series of 'contracts' which allow the data encapsulation of the logical world to be implemented in the physical. Each contract is developed in a semi-automated manner using data dictionaries. One

part of BOS is specially designed for high throughput requirements — the volume loading application (VLA) tackles the problem of loading millions of events received from the SDPs into DB2 in an efficient fashion. VLA is responsible for account assignment and usage limit monitoring.

The utilities layer provides a lot of the communications facilities needed for MSBP (see section 4.2 for details).

The applications layer in the CBP contain three of the four engines used in the bill run pipe — the import/export engine (IEE), the invoice production engine (IPE), and the bill delivery engine (BDE). IEE loads data from order-handling systems into the CIW. It is also the engine which provides the ledgering and revenue monitoring systems with the base data which they need. It allows any MIS system to define the timeliness and scope of the billing data it requires. However, its main job is to extract data out of the CIW for a bill run. IEE uses the utilities to prepare work unit files which conform to MSBP. The bill run pipe is composed of the following sections:

- import/export engine (IEE),
- pricing and assembly engine (PAE),
- invoice production engine (IPE),
- bill delivery engine (BDE),
- bill delivery manager (BDM).

IEE sends files of work units to a free PAE (which runs on adjunct Unix processors). The PAE passes its completed work units back into IPE. IPE applies bill reporting and composition rules to the work unit. These rules create an abstract description of the bill package, described in a syntax called the MIFI (media-independent formatting interface). BDE takes the MIFI description and creates media-specific invoices (for EDI, CD ROM, paper, etc). BDE replaces all the text identifiers with real language strings. The BDM oversees the distribution of the prepared invoices to production factories or special handling workbenches. The engines' memory management and data moving is performed by an application named the engine primer service (EPS). EPS plays a key role during the restart of engines in the event of major failure.

The last two major applications in the CBP are the receipting and debt management sub-system (RDM) and the billing data formatter (BDF). The architectural view of RDM is that it should be an engine like the rest; PDB should supply it with payment apportionment, follow-up, and behavioural scoring rules. However, MSBS uses a bought-in package for receipting. Receipting is one of the few areas in billing which is easy to bound. The integration has been designed so that the firewalls around the package make it

look like an engine to the rest of the system. The BDF application is used to receive bills from other external billing systems. The CBP will be deployed initially as a post-billing consolidator which supports cross-product discounting. BDF receives the bills from all BT's main billing systems and disassembles them into events. It also handles the financial handshakes involved in debt transfer between MSBS and the source billing systems.

The CBP is controlled by three major components — the process control and scheduler (PCS), the business procedure controller (BPC), and the exception management controller (EMC). PCS triggers the applications in the correct sequence for all the pipes defined in MSBS. As MSBS applications are multi-streamed, there can be many pipes operating in parallel. PCS is triggered by time, job completion, condition codes, dataset triggers, and NetView. Restart/recovery actions [12] are invoked automatically under predefined conditions. BPC is responsible for all the interactive transactions invoked from the BFOP. It works in conjunction with the work-flow manager in CRS. EMC handles all application exceptions in the CBP. It plays the role of a CBP 'agent' in the overall MCA. It manages error resolution queues, creates management reports, and provides a suite of automation platforms, work allocation managers, and decision support tools to implement the exception management business processes.

5.2 Pricing and assembly engine

Each CBP will be linked to a few adjunct SMP Unix machines. The machines run the PAE application. PCS provides IEE with the set of customers to be billed in each work unit. IEE creates the MSBP-conformant work files and the utilities transfer them to a free PAE. The processing windows of the PAE have been designed so that, once each pipe fills up, the mean PAE processing time and the mean IEE-IPE time is almost the same. The work unit files contain two main types of data — events and inventory. Events are held in the PAE's shared memory [13] (usually 4GB) in a list called the master event list (MEL). Each event has a link to the product instance in the inventory which generated it.

PDB supplies PAE with BT's product data. The information is in the form of a directed acyclic graph (DAG) of rule schemas. Every node and link in the DAG is timestamped. This sort of structure is also used in the work unit file to describe the customer's inventory. The customer graph is called the pricing hierarchy and its nodes and links describe the history of the customer's inventory. The PAE begins its processing by overlaying the customer and product graphs — a procedure which is termed 'lighting up the graph' — as shown in Fig 13. All the events are pulled through the lit-up graph from leaf nodes to the root. The

characteristics of the DAG ensure that each event has only one path through the graph.

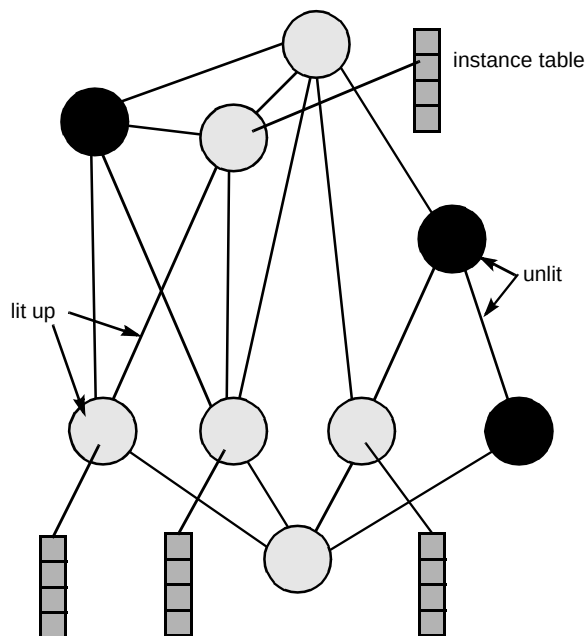


Fig 13 Directed acyclic graph processing.

The two main physical processes in the PAE are the billing and pricing process (BAP) and the rule interpreter process (RIP). Pools of both types of process are created in stand-by mode, where they use minimum CPU and memory resources. An overall control process will direct a BAP to process a particular customer. The BAP is given the whereabouts of the customer data and is provided with a shared memory segment. The BAP uses the customer data to light up the product graph. Each time a rule is encountered during the execution of the graph, the BAP asks the RIP control (RIPC) process for the services of a RIP to operate in its memory segment — as shown in Fig 14. So the pool of RIPs can potentially be running rules for many customers. If the PAE is processing a very large customer, it will use a single BAP and a large pool of RIPs. If it is processing a large number of small customers, the sizes of the BAP and RIP pools will vary. This flexibility point fulfils the requirement for a single solution to handle all customer segments.

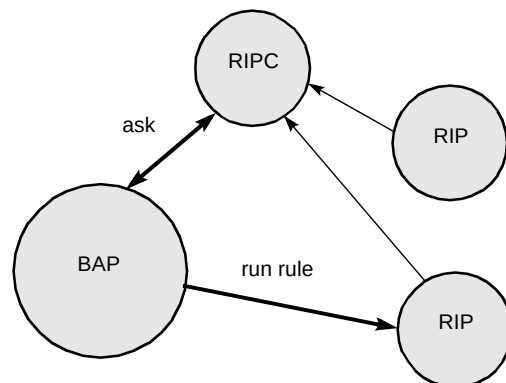


Fig 14 PAE physical processes.

The design process is all about trade-offs. There are advantages and disadvantages with the notion of an 'outboard' engine like the PAE. The disadvantages are those of any distributed system — commissioning and operating a monolithic system requires less organisational learning than the effort needed to integrate distributed, heterogeneous platforms. Performance has proven not to be an issue. The FDDI link approaches channel speeds, and therefore, by ensuring that the work units are of the right size, the overhead of transferring data in and out is minimal. The advantages are considerable. It is the considered view that the technology described above could not realistically have been developed on a mainframe. However, the perceived risks of building the CIW on Unix were too great, so a complete separation of the warehouse from the PAE was necessary. The cost benefits of Unix systems are also a factor. Perhaps the most important advantage of the architecture will be realised in the maintenance phase; the imposition of a physical boundary between components is a crude but highly effective way of retaining modularity.

5.3 Portfolio database

PDB will hold the rules which define BT's entire portfolio of products. The MSBS architecture envisages that many of the components (such as the PAE) will be deployed in a product development environment as well as the mainstream operational one. PDB's development environment will be used by product designers to model and test new products. PDB is designed to support the entire product launch process. For example, it has to generate the pricing manuals which go to OfTel and move the products through the development and approval phases before they are cascaded down to the operational systems.

PDB replaces a whole range of product repositories which existed throughout the business. The systems to which the older repositories downloaded data still need to be driven. So PDB has created 'gateways' between the legacy system's individual approaches to product coding and the MSBS product model. These gateways are a vital component in the migration of product sets on to MSBS.

PDB uses Oracle Version 7 and Forms 4.5 to build the workbenches which product designers use. These workbenches will hide the complexity of rules syntaxes and product relationships from the designer. However, they do assume that product designers are technically literate analysts. The notion that products can be created directly by product managers from a GUI is unrealistic.

5.4 Management and control architecture

MSBS has pioneered the idea of a single management and control architecture (MCA) for BT's operational support systems [14]. The MCA covers the main disciplines of:

- business management (accounting, security, and user management),
- performance management,
- problem management,
- operations management (lights-out computing managed by command centres),
- configuration management,
- change management,
- income integrity.

The architecture is shown schematically in Fig 15.

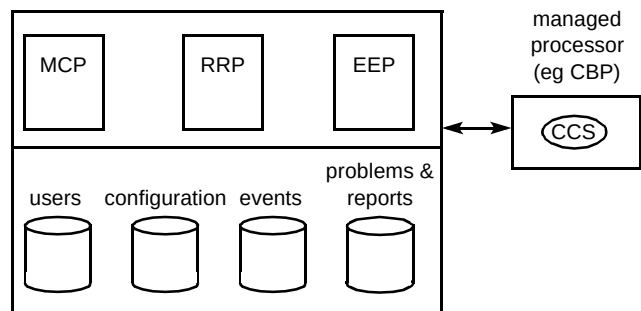


Fig 15 Management and control architecture.

MCA is based on a manager/agent paradigm. A management and control bridge (MCB) communicates with component control sub-systems (CCS) on each processor. Some of the applications which make up CCS have already been mentioned — the CBP's exception management controller (EMC) is a good example. The job of the MCB is to monitor and control the end-to-end activity of MSBS at all levels of MSBP. The MCB contains the third warehouse in MSBS — the operational repository. The servers which it contains are shown in Fig 15. The applications are actually clustered processors — the record reconciliation processor (RRP), the event error processor (EEP), and the monitor and control processor (MCP). The RRP monitors and reports on all income integrity matters. The EEP controls the handling of errored events in the main processors. It sets policy rules for the automation platforms in the three main processors and reports on the effect of errors on unbilled revenue. EEP also handles write-offs. The MCP is responsible for starting and stopping processors and applications. MCP has standardised conventions for SNMP alarms to which each CCS must adhere. MVS/Unix integration is achieved using IBM's OSM/6000 product.

The scale of MSBS makes the MCA a critical success factor; asset management of thousands of PCs, distribution of client software, LAN installation and maintenance, and provision of help desks and user training are all bread-and-butter issues, but they have the potential to consume an inordinate amount of management time unless the system support for the new processes has been carefully designed.

5.5 Service domain processor

The SDP is a large SMP Unix machine which runs the same pricing engine as the PAE. It is used in the POTS world to perform initial customer-specific transaction pricing. The SDP receives files of unpriced PIR-encoded events from the NMP. It prices them and routes them to the correct CBP. The philosophy behind the SDP is to do as much processing before the bill run starts as possible. The batch window for billing is always critical, even for a 24 hr/7 day system like MSBS. Given that the vast majority of products reduce to volume-related discounts applied to transaction priced events, it is a great help to the engineer if tariffing need not be performed during the bill run. The SDP 'smoothes out' the flow of data into the CIW to ensure that there are no huge peaks and troughs in the VLA's workload. Customer-specific routing and tariffing inventory are downloaded from the CIW to the SDPs using MSBP. PDB provides the tariffing rules in the same manner as it delivers bill-time product rules to the PAE. (The PAE has the capability to do basic tariffing, in case a product requires actual re-tariffing of events based on volume or a configuration management exception arises within the SDP.)

The relative cost of billing is rising rapidly. It is not unlikely that the economics of processing, storing and reporting certain categories of usage will lose their validity. The most likely scenario is that some customers will be offered subscription-based products rather than usage-based tariffs. The SDP, which holds customer information, is an ideal point in the architecture for filtering to occur.

There will be SDPs deployed for each of BT's major services (PSTN, Cashless services, and BT's VPN offerings, for example). It plays an important role in the migration of services on to MSBS. In the first phase, SDPs are deployed as preprocessors to existing billing systems, shown in Fig 16. As customers are migrated onto the MSBS, the SDPs are instructed to start routing usage for these customers into the appropriate CBP.

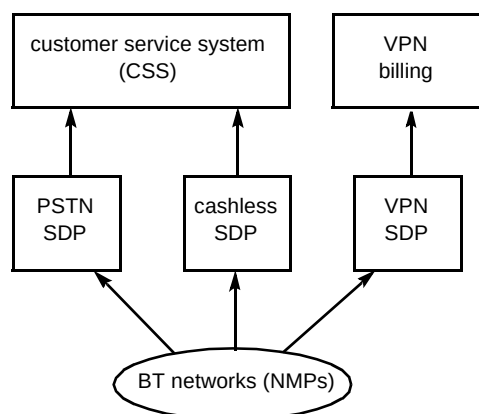


Fig 16 Initial SDP deployment.

This phase of MSBS has already been completed. A version of the SDP (data driven by a precursor of PDB) was used to take the unit-based charging off BT's 6000 PSTN exchanges. Twelve SDPs handle all of BT's PSTN usage — about 120M calls each night. This engineering achievement led to Billing 90s very successful 'per second pricing' project, which increased the number of people in the world who receive elapsed time charging from 20M to 50M overnight.

In the IN world, the SDP will run as a portable application on an IN platform. PDB's vision of total service creation will link the IN service rules with the real-time pricing rules.

5.6 Network mediation processor

The NMP is responsible for the collection, transformation, transfer, routing, and validation of usage events. It polls the PDGs for which it is responsible and collects events. The events are in a variety of PDG-specific formats. The NMP then uses PDB translation rules to create the PIR-encoded events defined in the product model. The NMP routes events, based on feature ranges, to SDPs using BDS. Validation and auto-edit rules are supplied by another system called the network model database (NMDB). The NMP uses a software bus to link application servers responsible for monitoring, filtering and securing the data.

5.7 Front office

The BFOP is a suite of applications responsible for hierarchy editing, bill view and amend, and receipting functions. The common CRS concept is not yet in place, so MSBS has been forced to implement its own supporting infrastructure. The applications use BPC to control their access to the BOS contracts which retrieve and store the BFOP's data.

The BFOP is the only genuinely OO implementation in MSBS. The infrastructure provides object cache managers, contract disassemblers, and tokenisers. The hierarchy editor is a GUI which allows cost centres to be created and moved graphically. Actual order taking for billing products is performed by an existing BT system called COSMOSS [15]. The COSMOSS screens for these products are created automatically by PDB rules which are downloaded to COSMOSS. This capability allows billing products to be implemented without any code being written.

The BFOP is not the only front-office component — 'BT DeskTop' is an application which is provided to customers. Indexed billing data is provided to these customers on CD ROM. The customer uses the 'DeskTop' application to create bespoke reports or to perform data mining. MSBS supports an alternative deployment of 'DeskTop' on secure Intranets. Customers will pay for the

ability to request a range of reports over their IntraNet. (MSBS views the Intranet server as a PDG.) In the longer term these access mechanisms provide a potential reduction in postage costs for BT.

5.8 Performance issues

No discussion of an engineering project the size of MSBS would be complete without a reference to performance tuning. Many of the machines used in the design are large Unix SMP boxes. A key aspect of the design, therefore, is to ensure that the addition of extra processors has a proportional effect on performance. Some results from PAE performance testing are shown in Fig 17. The balance of RIPs for various numbers of CPUs can really only be made after looking at these sorts of results. Once a reasonable number of CPUs are in place, the effect of parallelism is quite dramatic. The results provide reassurance that the design fully exploits parallelism and is therefore scalable.

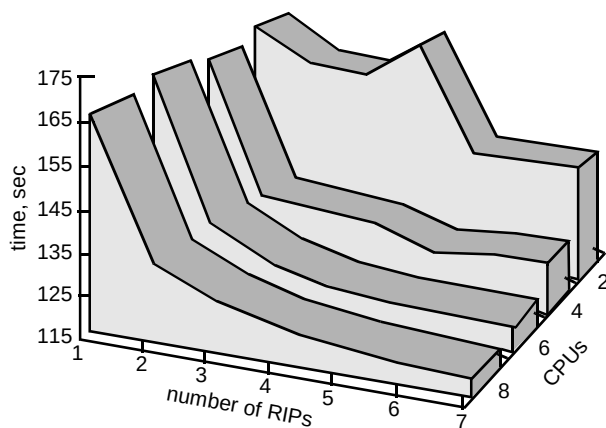


Fig 17 Processor tuning.

6. Conclusions

The sheer scale and complexity of the problem which MSBS seeks to solve is formidable. The design's premise — that multiple services can be described using a single, coherent, set of abstractions — was a bold one. The creation of these abstractions provides a good case study on the role of computer science in the design of commercial systems. However, it is the software engineer's job which is most challenging — the production engineering of this flexible platform, deployed on such a huge scale, is so complicated that its construction has few parallels in industry today.

A telco's billing system — more than its networks — provides the armoury for the competitive battles which lie ahead. Its construction is usually a long and painful experience, but the prize awarded to those who stay calm under fire is a great one.

References

- 1 Luftman J N et al: 'Transforming the enterprise: the alignment of business and information technology strategies', *IBM Systems Journal*, **32**, No 1 (1993).
- 2 Whyte W S: 'The many dimensions of multi-media communications', *BT Technol J*, **13**, No 4, pp 9—20 (October 1995).
- 3 Anand T S et al: 'Turning customer data into dollars', *AT&T Technology*, **10**, No 4 (1995/1996).
- 4 Mathieson G: 'Customer handling intermediate server — an architecture-led project', *BT Technol J*, **15**, No 1, pp 114—134 (January 1997).
- 5 Mili H et al: 'Reusing software: issues and research directions', *IEEE Transactions on Software Engineering*, **21**, No 5 (June 1995).
- 6 Morgan M J et al: 'Service creation technologies for the intelligent network', *AT&T Technical J*, **70**, Nos 3/4 (Summer 1991).
- 7 Harrison P F: 'Customer service system — past, present and future', *BT Technol J*, **15**, No 1, pp 29—45 (January 1997).
- 8 Furley N: 'The BT operational support system architecture', *BT Technol J*, **15**, No 1, pp 13—21 (January 1997).
- 9 Cordingley E S and Dai H: 'Encapsulation — an issue for legacy systems', *BT Technol J*, **11**, No 3, pp 52—64 (July 1993).
- 10 Satyanarayanan M: 'The influence of scale on a distributed file system', *IEEE Transactions on Software Engineering*, **18**, No 1 (January 1992).
- 11 Roussopoulos N et al: 'An architecture for high performance EIS', *IEEE Transactions on Software Engineering*, **17**, No 1 (January 1991).
- 12 Nichola V: 'Comparative analysis of different models of checkpointing and recovery', *IEEE Transactions on Software Engineering*, **16**, No 8 (August 1990).
- 13 Levelt W G et al: 'Comparison of two paradigms for distributed shared memory', *Software Practice and Experience*, **22**, No 11 (November 1992).
- 14 Bouch S M, Hayes I C and Oldham T J: 'Systems management of operational support systems applications', *BT Technol J*, **15**, No 1, pp 151—159 (January 1997).
- 15 Chandler J: 'Management of special services — designing for a changing world', *BT Technol J*, **15**, No 1, pp 87—97 (January 1997).



Jim Crookes graduated from Queen's University, Belfast, in 1985 with a degree in Physics and Computer Science. He joined BT's Belfast Engineering Centre (BEC) to work on data communications and network management systems. He was a member of the Billing 90s design team, and in 1994 was appointed Billing 90s Chief Architect. Until 1996 he was BT's design authority for the MSBS development. He is now a project manager in BEC, working on large-scale billing developments.